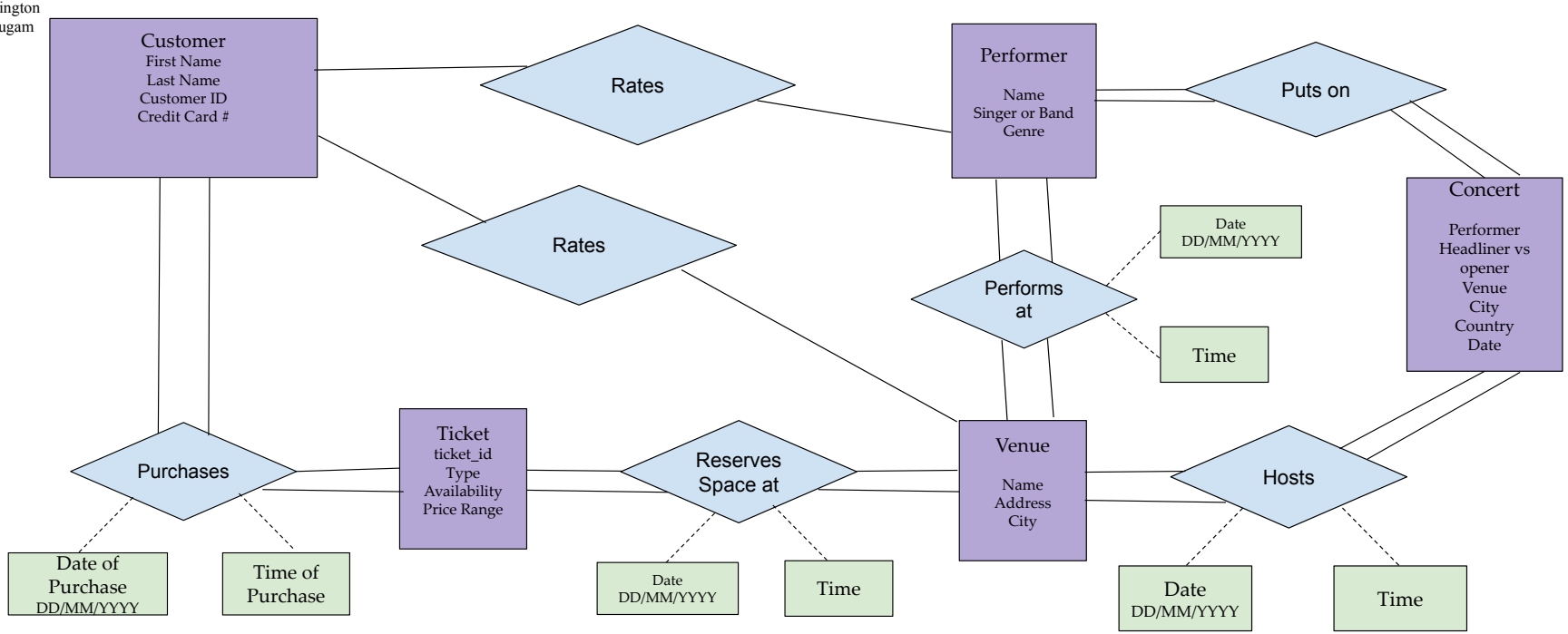
Destiny Brewington
Divya Shanmugam




```
Create Table Customer (
  customer_id integer,
  Primary Key(customer_id)
  credit_card_num varchar(16) not null,
  Firstname TEXT not null,
  Lastname TEXT not null,

)
```

```
Create Table Tickets (
  ticket_id Integer,
  Primary Key (ticket_id),
  type TEXT,
  Foreign Key customer_id references Customer,
  Foreign Key (venue_name, venue_address) references Venue
  num_tix_avail() Integer,
  Check num_tix_avail() > 0,
  num_tix_puchased,
  price_range( )
)
```

```
Create Table Performer (
  performer_ID Integer
  performer_name TEXT,
  Primary Key (performer_id, performer_name)
  singer BOOLEAN not null,
  band BOOLEAN not null,
  genre TEXT not null,
)
```

```
Create Table Venue (
  venue_name TEXT,
  venue_address TEXT,
  Primary Key (venue_name, venue_address),
  city TEXT not null,
  country TEXT not null,
)
```

```
Create Table Concert (
  performer_name TEXT not null,
  headliner BOOLEAN not null,
  opener BOOLEAN not null,
  venue_name TEXT not null,
  city TEXT not null,
  country TEXT not null,
  Foreign Key (performer_id, performer_name) references (Performer)
  Foreign Key (venue_name, venue_address) references (Venue)
  date DATE not null,
)
```

The application will resemble ticket-vending sites like Ticketmaster and SeatGeek, allowing users to access information on upcoming concerts and favorite artists. Its domain, *Concerts*, includes six entities: Performer, Venue, Date, Event, Ticket, and Rating, each with attributes forming six entity sets and six relationships.

A Performer performs at a Venue, with Date and Time as attributes of the *Performs At* relationship. This relationship is many-to-many, with required participation (only performers with scheduled venues are stored). Customer and Ticket also have a required relationship, as the database only stores customers who purchased tickets. Similarly, Ticket and Venue connect through *Reserves Space At*. The *Rates* relationship between Customer and Rating does not require participation of either entity, as ratings are optional. Overall, all relationships are many-to-many, reflecting the variability among entities. Data will be sourced from Ticketmaster and SeatGeek's public API, which provides details such as start times, venues, and ticket prices.

The project will be expanded through a Web Front-End featuring three main functions: search, rate, and purchase. On the homepage, users can search for an artist in order to access their homepage. Each artist's page will display a list of their upcoming concerts, their average rating (from Ticketmaster), and a button to submit a new rating, which will update the artist's overall score in the database. When a user clicks on an upcoming concert, they will be directed to a separate webpage displaying details regarding the concert's date, pricing, and venue seat map. On that webpage, a user can "select a seat" and "purchase" a ticket. After a ticket purchase, the application will generate a unique customer ID and add the transaction to the database.

Should a member of the team drop the class, the remaining team member will only create the search function for the web front.